

DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 10:

More MPI

Recap of previous MPI lecture

Many parallel programs can be written using just these six functions:

- `MPI_Init`
- `MPI_Finalize`
- `MPI_Comm_size`
- `MPI_Comm_rank`
- `MPI_Send`
- `MPI_Recv`

Message passing is a simple programming model, but there are some special issues

- Buffering
- deadlock
- Deterministic execution
- Performance

Communication performance

- Basic info: *latency* and *bandwidth*
- Simplest model: $t_{\text{comm}} = \alpha + \beta M$
- More realistic: CPU overhead ...
- Different networks have different parameters
- Can tell a lot via a simple ping-pong experiment

OpenMPI on Bridges-2

- How many chips per nodes?
- Heterogeneous network:
 - Crossbar switch between cores (?)
 - Bus between chips (?)
 - Gigabit ethernet between nodes (?)
- Process layout (16 process example)
 - Processes 0-3 on first chip, first node
 - Processes 4-7 on second chip, first node
 - Processes 8-11 on first chip, second node
 - Processes 12-15 on second chip, second node
- Test ping-pong from 0 to 1, 7, and 8.

Performance model

- Simplest model: $t_{\text{comm}} = \alpha + \beta M$
- More realistic: CPU overhead ...
- Different networks have different parameters
- Can tell a lot via a simple ping-pong experiment

On Bridges-2:

Approximate α - β parameters (on node)

Approximate α - β parameters (cross-node)

Reminder: basic send and recv

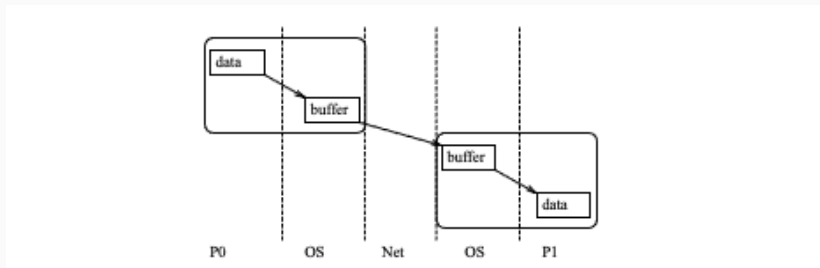
```
MPI_Send(buf, count, datatype,  
         dest, tag, comm);
```

```
MPI_Recv(buf, count, datatype,  
         source, tag, comm, status);
```

MPI_Send and MPI_Recv are *blocking*

- Send does not return until data is in system
- Recv does not return until data is ready

Blocking and buffering



Block until data “in system” — maybe in a buffer:

- The user must NOT overwrite the send buffer until the send (data transfer) is complete.
- The user can NOT use the receiving buffer before the receive is complete.

Alternative: don't copy, block until done.

Problem 1: Potential deadlock

Send a large message from process 0 to process 1. If there is insufficient storage at the destination, the send must wait for the user to provide the memory space (through a receive).

Process 0	Process 1
Send (1)	Send (0)
Recv (1)	Recv (0)

Both processors wait to finish send before they can receive!

May not happen if lots of buffering on both sides.

It depends on the availability of system buffers in which to store the data sent until it can be received.

Solution 1: Alternating order

Could alternate who sends and who receives.

Process 0	Process 1
Send (1)	Recv (0)
Recv (1)	Send (0)

Combined send/recv: Supply receive buffer at same time as send.

Process 0	Process 1
Sendrecv (1)	Sendrecv (0)

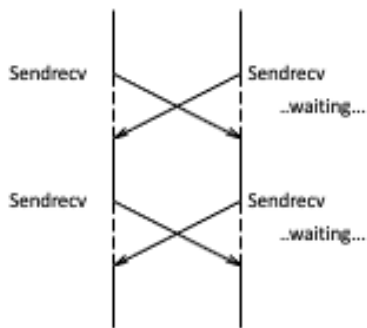
Combined sendrecv

Allows simultaneous send and receive.

```
MPI_Sendrecv(sendbuf, sendcount, sendtype,  
              dest, sendtag,  
              recvbuf, recvcount, recvtype,  
              source, recvtag,  
              comm, status);
```

Combining send and recv to avoid deadlock.

Problem 2: Communication overhead



Partial solution: nonblocking communication

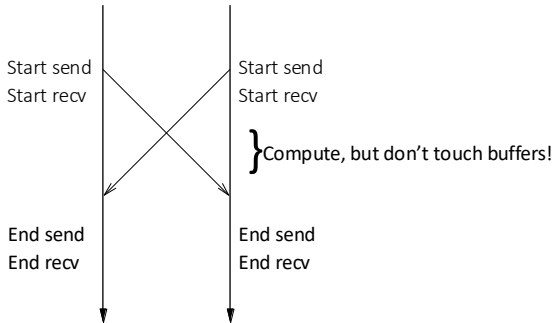
Blocking vs non-blocking communication

- `MPI_Send` and `MPI_Recv` are *blocking*
 - Send does not return until data is in system
 - Recv does not return until data is ready
 - Cons: possible deadlock, time wasted waiting
- Why blocking?
 - To guarantee not to overwrite buffer during send.
 - To guarantee not to read buffer before data ready.
- Alternative: *nonblocking* communication
 - Split into distinct initiation/completion phases
 - Initiate send/recv and promise not to touch buffer
 - Check later for operation completion

The default Sends try not to block, but don't guarantee it. As your data structures grow large, this blocking behavior may well emerge.

Overlap communication and computation

Course: DS-642



Nonblocking operations

Initiate message:

```
MPI_Isend(start, count, datatype, dest  
          tag, comm, request);  
MPI_Irecv(start, count, datatype, dest  
          tag, comm, request);
```

Wait for message completion:

```
MPI_Wait(request, status);
```

Test for message completion:

```
MPI_Test(request, status);
```

Nonblocking operations

Initiate message:

```
MPI_Isend(start, count, datatype, dest  
          tag, comm, request);  
MPI_Irecv(start, count, datatype, dest  
          tag, comm, request);
```

- The user must NOT overwrite the send buffer until the send (data transfer) is complete.
- The user can NOT use the receiving buffer before the receive is complete.

Nonblocking operations

Initiate message: Wait for message completion:

```
MPI_Wait(request, status);
```

Test for message completion:

```
MPI_Test(request, status);
```

- “request” is used to identify a previously posted send/receive
- `MPI_Wait()` returns when the operation is complete, and the status is updated for a receive.
- `MPI_Test()` returns immediately, with “flag” = true if posted operation corresponding to the “request” handle is complete.

Multiple outstanding requests

Sometimes useful to have multiple outstanding messages:

```
MPI_Waitall(count, requests, statuses);  
MPI_Waitany(count, requests, index, status);  
MPI_Waitsome(count, requests, indices, statuses);
```

Multiple versions of test as well.

Other send/recv variants

Other variants of `MPI_Send`

- `MPI_Ssend` (synchronous) – do not complete until receive has begun (unsafe programs deadlock).
- `MPI_Bsend` (buffered) – user provides buffer (via `MPI_Buffer_attach`) – user allocates enough memory to make an unsafe program safe.
- `MPI_Rsend` (ready) – user guarantees receive has already been posted.
- Can combine modes (e.g. `MPI_Issend`)

`MPI_Recv` receives anything.

Another approach

- Send/recv is one-to-one communication
- An alternative is one-to-many (and vice-versa):
 - Collective routines provide a higher-level way to organize a parallel program
 - Each process executes the same communication operations
 - MPI provides a rich set of collective operations
 - *Broadcast* distributes data from one process (the root) to all others in a communicator
 - *Reduce* combines data from all processes in communicator and returns it to one process
 - Operations are called by all processes in communicator

In many numerical algorithms, send/recv can be replaced by broadcast/reduce, improving both simplicity and efficiency

Classification by Operation

- One-To-All Mode
 - One process contributes to the results.
 - All processes receive the result.
 - `MPI_Bcast()`, `MPI_Scatter()`, `MPI_Scatterv()`
- All-To-One Mode
 - All processes contribute to the result.
 - One process receive the result.
 - `MPI_Gather()`, `MPI_Gatherv()`, `MPI_Reduce()`
- All-To-All Mode
 - All processes contribute to the result.
 - All processes receive the result.
 - `MPI_Alltoall()`, `MPI_Allgather()`,
`MPI_Allreduce()`
- Collective operations that do not fit into above categories
 - `MPI_Scan()`, `MPI_Barrier()`

Broadcast and reduce

```
MPI_Bcast(buffer, count, datatype,  
          root, comm);  
MPI_Reduce(sendbuf, recvbuf, count, datatype,  
          op, root, comm);
```

- `buffer` is copied from root to others
- `recvbuf` receives result only at root
- $op \in \{ \text{MPI_MAX}, \text{MPI_SUM}, \dots \}$

Predefined reduction operators in MPI

Operation Value	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical and
MPI_BAND	Bitwise and
MPI_LOR	Logical or
MPI_BOR	Bitwise or
MPI_LXOR	Logical exclusive or
MPI_BXOR	Bitwise exclusive or
MPI_MAXLOC	Maximum and location of maximum
MPI_MINLOC	Minimum and location of minimum

Example: π

```
#include <mpi.h>
#include <math.h>
#include <stdio.h>

void main(int argc, char** argv) {
    int n, rank, nproc, i;
    double istart, iend, sum, pi;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nproc);
    if (rank == 0) {
        printf("Enter the number of intervals: (0 qui
        scanf("%d",&n);
    }
```

Example: π

```
MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
istart = 1 + n*((rank + 0.0)/nproc);
iend = n*((rank + 1.0)/nproc);
sum = 0.0;
for (i = istart; i <= iend; ++i)
    sum += 0.5/((i-0.75)*(i-0.25));
MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0,
if (rank == 0) {
    printf("processes = %d, n = %ld, pi = %f\n", n
    }
MPI_Finalize();
}
```


Benchmarking Parallel Performance

`double MPI_Wtime(void)`: Returns an elapsed time in seconds on the calling processor.

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    double t1, t2;
    MPI_Init(&argc, &argv);
    t1 = MPI_Wtime();
    sleep(1);
    t2 = MPI_Wtime();
    printf("MPI_Wtime measured a 1 second sleep to ");
    fflush(stdout);
    MPI_Finalize();
    return 0;
}
```

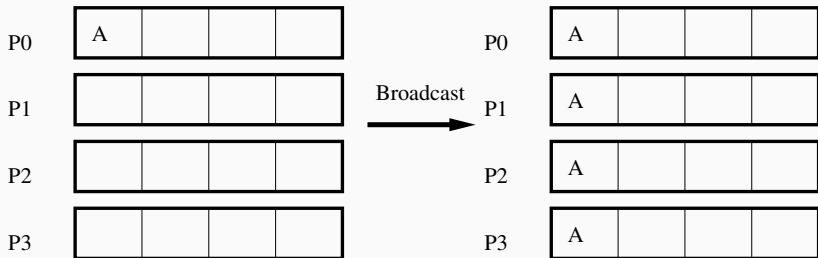
Synchronization: Barrier

Blocks until all processes in the group of the communicator `comm` call it.

```
MPI_Barrier(comm);
```

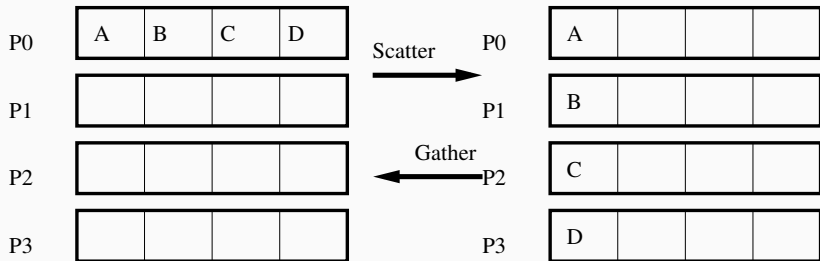
Not needed that often. Occasionally useful in measuring performance and load balancing.

Broadcast

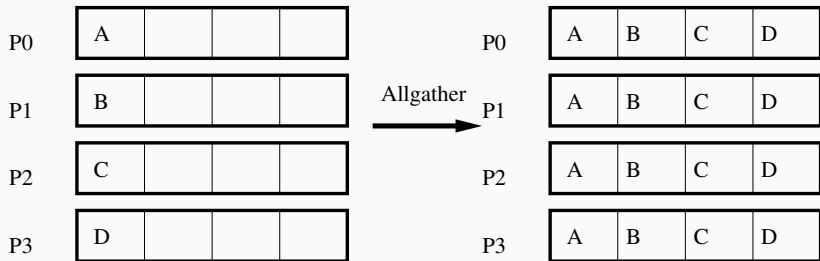


- All collective operations must be called by all processes in the communicator.
- `MPI_Bcast` is called by both the sender (called the root process) and the processes that are to receive the broadcast.
 - “root” argument is the rank of the sender; this tells MPI which process originates the broadcast and which receive

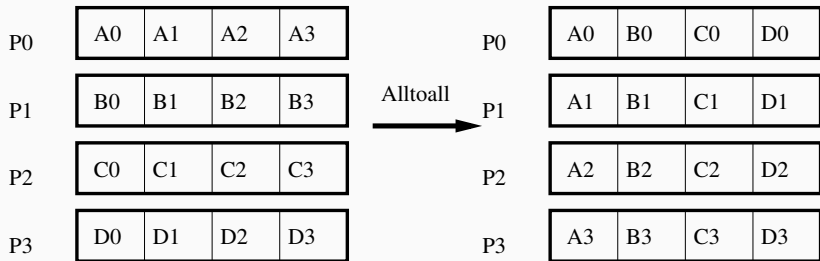
Scatter/gather



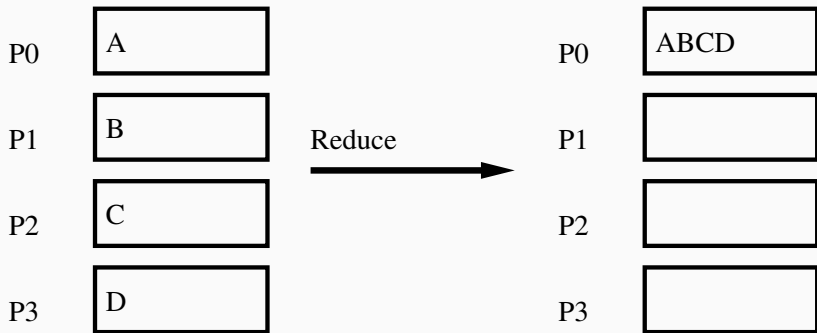
Allgather



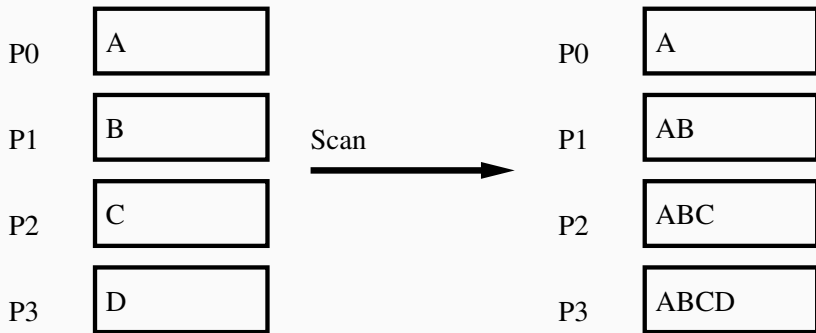
Alltoall



Reduce



Scan



MPI Collective Routines

- **Many Routines:** `Allgather`, `Allgatherv`, `Allreduce`, `Alltoall`, `Alltoallv`, `Bcast`, `Gather`, `Gatherv`, `Reduce`, `Reduce_scatter`, `Scan`, `Scatter`, `Scatterv`
- (`v` suffix) vector variants - allow the chunks to have variable sizes.
- All variants, for example, `Allreduce`, `Reduce_scatter` deliver results to all participating processes.
- MPI-2 adds `Alltoallw`, `Exscan`, intercommunicator versions of most routines.
- MPI-3 adds one-sided communication.

MPI Collective Routines

- Mostly used routines:
 - `Init/Finalize`
 - `Get_comm_rank, Get_comm_size`
 - `Send/Recv` **variants and** `Wait`
 - `Allreduce, Allgather, Bcast`